Unidad 2:
Scripts de Administración y Automatización en el SGBD

FICHA TÉCNICA DE LA UNIDAD

Unidad Trabajo	UD 2: Scripts de Administración y Automatización en el SGBD
Resultados de Aprendizaje Asociados	**RA3:** Automatiza tareas de administración del sistema gestor, realizando guiones y programando tareas.
Criterios de Evaluación	CE 3.a, CE 3.b, CE 3.c, CE 3.d
Tiempo	12 horas lectivas (equivalente a unas 4 semanas de clase en el módulo ASGBD)

ÍNDICE DE CONTENIDOS

1. Repaso Teórico

1.1. Procedimientos Almacenados (Stored Procedures)

1.2. Funciones Almacenadas (Stored Functions)

1.3. Disparadores (Triggers)

2. Ventajas de la programación en la administración

3. Catálogo del Sistema (Metadatos)

3.1. information_schema (El Diccionario de Datos)

3.2. Base de Datos mysql (El Repositorio de Seguridad y Privilegios)

3.3. Base de Datos performance_schema

3.4. Base de Datos sys (La Abstracción Simplificada)

4. Automatización Mediante Scripts Externos (Bash)

5. Ejemplos Prácticos de Scripts de Administración (Sencillos y Documentados)

1. Repaso Teórico

La administración moderna de bases de datos no se limita a ejecutar consultas interactivas. Para garantizar la eficiencia, seguridad y automatización, los SGBD permiten almacenar lógica programada directamente en el servidor.

A continuación, repasamos los tres pilares de la programación en base de datos: **Procedimientos Almacenados**, **Funciones** y **Disparadores (Triggers)**.

1.1. Procedimientos Almacenados (Stored Procedures)

Son subprogramas que se compilan y almacenan en el SGBD. Pueden realizar operaciones complejas, alterar datos en múltiples tablas y ejecutar sentencias DDL (Data Definition Language).

- **Parámetros:**
  - IN (Entrada): El valor se pasa al procedimiento pero no se puede modificar hacia el exterior.
  - OUT (Salida): El procedimiento asigna un valor a este parámetro para devolverlo al entorno que lo invocó.
  - INOUT (Entrada/Salida): El parámetro aporta un valor inicial y puede ser modificado dentro del procedimiento para devolver el nuevo estado.
- **Invocación:** Se ejecutan explícitamente mediante la instrucción CALL nombre_procedimiento(parametros);.
- **Uso Administrativo:** Ideales para tareas de mantenimiento repetitivas (ej. creación masiva de usuarios, particionamiento de tablas, purgas programadas de logs).

Ejemplo de Sintaxis Básica (MySQL)

```
DELIMITER //
CREATE PROCEDURE sp_crear_usuario_lectura(
IN p_username VARCHAR(50),
IN p_password VARCHAR(50),
IN p_db VARCHAR(50)
```

```
)
BEGIN
  -- Generar la sentencia dinámicamente y ejecutarla de forma segura
  SET @sql_create = CONCAT('CREATE USER "', p_username, '"@"localhost" IDENTIFIED BY "', p_pa
ssword, '";');
  PREPARE stmt1 FROM @sql_create;
  EXECUTE stmt1;
  DEALLOCATE PREPARE stmt1;
  SET @sql_grant = CONCAT('GRANT SELECT ON ', p_db, '.* TO "',p_username, '"@"localhost";');
  PREPARE stmt2 FROM @sql_grant;
  EXECUTE stmt2;
  DEALLOCATE PREPARE stmt2;
  FLUSH PRIVILEGES;
END //
DELIMITER ;
```

1.2. Funciones Almacenadas (Stored Functions)

A diferencia de los procedimientos, las funciones están diseñadas específicamente para **calcular y devolver un único valor escalar** (un número, una cadena de texto, una fecha).

- **Características Clave:**
  - Deben contener obligatoriamente al menos una cláusula RETURN.
  - **No** pueden invocar sentencias de transacción (COMMIT o ROLLBACK) ni modificar directamente el estado de las tablas en la mayoría de escenarios de consulta estándar.
  - Se pueden integrar directamente dentro de sentencias SQL estándar (SELECT, WHERE, ORDER BY).
- **Uso Administrativo:** Ideales para normalizar formatos, calcular métricas rápidas de rendimiento o formatear salidas del catálogo.

Ejemplo de Sintaxis Básica (MySQL)

```
DELIMITER //
CREATE FUNCTION fn_obtener_tamano_megas(p_bytes BIGINT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
  RETURN p_bytes / 1024 / 1024;
```

```
END //
DELIMITER ;

-- Uso de la función en una consulta administrativa:
SELECT table_name, fn_obtener_tamano_megas(data_length) AS 'Tamano_MB'
FROM information_schema.tables
WHERE table_schema = 'mi_base_datos';
```

1.3. Disparadores (Triggers)

Un disparador es un bloque de código que se ejecuta **de manera automática** en respuesta a un evento DML (INSERT, UPDATE o DELETE) sobre una tabla específica.

- **Estructura temporal:**
  - BEFORE: Se ejecuta antes de que los cambios se validen e inserten en la base de datos (útil para validación y saneamiento de datos).
  - AFTER: Se ejecuta después de que el cambio se ha confirmado en la tabla (ideal para auditorías, propagación de datos y sincronizaciones).
- **Variables Especiales:**
  - OLD: Accede a los valores de la fila antes del evento (disponible en UPDATE y DELETE).
  - NEW: Accede a los nuevos valores que se van a guardar (disponible en INSERT y UPDATE).
- **Uso Administrativo:** Monitorización de cambios críticos, auditoría de seguridad (saber quién modificó qué y cuándo) e integridad referencial compleja.

Ejemplo de Sintaxis de Auditoría (MySQL)

```
CREATE TABLE log_cambios_privilegios (
id INT AUTO_INCREMENT PRIMARY KEY,
usuario_afectado VARCHAR(100),
accion VARCHAR(20),
fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
realizado_por VARCHAR(100)
);

DELIMITER //
```

```
CREATE TRIGGER tg_auditoria_usuarios_update
AFTER UPDATE ON mysql.user
FOR EACH ROW
BEGININSERT INTO log_cambios_privilegios (usuario_afectado, accion,realizado_por)
VALUES (OLD.user, 'MODIFICACION_PERMISOS', USER());
END //
DELIMITER ;
```

2. Ventajas de la programación en la administración

El uso de estos elementos dentro del SGBD ofrece tres ventajas operativas críticas para un Administrador de Sistemas de Bases de Datos (ASGBD):

Ventaja	Descripción Técnica	Impacto en la Administración
Reutilización y Mantenimiento	Encapsula la lógica de negocio y administración. Si un proceso de backup cambia, solo se modifica el objeto en el servidor, no las aplicaciones externas.	Menor duplicación de código y reducción del error humano.
Seguridad de Acceso	Permite restringir el acceso directo a tablas físicas mediante permisos GRANT EXECUTE sobre procedimientos específicos.	El usuario opera la BD a través de "cajas negras" controladas.
Rendimiento de Red	Reduce el tráfico de red enviando únicamente la orden CALL en lugar de miles de sentencias SQL secuenciales.	Ejecuciones mucho más rápidas en redes congestionadas o entornos cloud.

3. Catálogo del Sistema (Metadatos)

MySQL almacena la información sobre su propia estructura (metadatos) en esquemas específicos. Dominar estos catálogos es la herramienta más potente de un Administrador de Sistemas (SysAdmin), ya que permite auditar el servidor, monitorizar el almacenamiento y diagnosticar la degradación de rendimiento directamente mediante sentencias SQL.

3.1. information_schema (El Diccionario de Datos)

Es una base de datos virtual de solo lectura definida por el estándar ANSI SQL. Actúa como el diccionario de datos del SGBD. No contiene archivos de datos físicos; en su lugar, MySQL genera sus tablas dinámicamente en memoria al consultarlas.

- **SCHEMATA:** Almacena información sobre las bases de datos existentes en el servidor (nombres, juegos de caracteres predeterminados, etc.).
- **TABLES:** Contiene metadatos de todas las tablas físicas y vistas (motor de almacenamiento, número estimado de filas, tamaño de los datos y de los índices).
- **COLUMNS:** Detalla cada columna de cada tabla (tipos de datos, nulabilidad, privilegios asignados).
- **STATISTICS:** Proporciona información de bajo nivel sobre los índices de las tablas del sistema.
- **ROUTINES:** Almacena información sobre los procedimientos y funciones del SGBD.

Caso Práctico 1: Obtener el tamaño en disco de cada base de datos

Esta consulta calcula el espacio real en disco de cada base de datos (Datos + Índices) expresado en Megabytes:

```
SELECT
table_schema AS 'Base de Datos',
ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Tamaño
total (MB)',
ROUND(SUM(data_length) / 1024 / 1024, 2) AS 'Solo Datos (MB)',
ROUND(SUM(index_length) / 1024 / 1024, 2) AS 'Solo Índices (MB)'
FROM information_schema.tables
GROUP BY table_schema
ORDER BY (SUM(data_length + index_length)) DESC;
```

Caso Práctico 2: Detectar tablas sin Clave Primaria (Primary Key)

La falta de claves primarias degrada catastróficamente el rendimiento, especialmente en entornos con replicación activa (Master-Slave) o clústeres.

```
SELECT
t.table_schema AS 'Base de Datos',
t.table_name AS 'Tabla sin PK'
```

```
FROM information_schema.tables t
LEFT JOIN information_schema.table_constraints tc
ON t.table_schema = tc.table_schema
AND t.table_name = tc.table_name
AND tc.constraint_type = 'PRIMARY KEY'
WHERE tc.constraint_name IS NULL
AND t.table_schema NOT IN ('information_schema', 'mysql',
'performance_schema', 'sys')
AND t.table_type = 'BASE TABLE';
```

3.2. Base de Datos mysql (El Repositorio de Seguridad y Privilegios)

A diferencia de information_schema, la base de datos mysql contiene tablas físicas persistentes en el disco. Es el cerebro de la seguridad de accesos del servidor.

- **user:** Almacena los usuarios creados, hosts permitidos, contraseñas cifradas y privilegios globales del sistema.
- **db:** Controla los privilegios específicos de los usuarios sobre bases de datos completas.
- **tables_priv y columns_priv:** Regulan accesos extremadamente granulares a nivel de tabla o columna.

Caso Práctico 3: Auditoría rápida de expiración de contraseñas de usuarios

Para cumplir con normativas de seguridad (como el RGPD), un SysAdmin debe controlar la antigüedad y vigencia de las cuentas del sistema:

```
SELECT
user AS 'Usuario',
host AS 'Host de Origen',
password_expired AS '¿Expirada?',
password_last_changed AS 'Último cambio',
password_lifetime AS 'Días de ciclo de vida'
FROM mysql.user
WHERE ssl_type != '' OR password_expired = 'Y' OR user !=
'mysql.infoschema';
```

3.3. Base de Datos performance_schema

Se enfoca de forma estricta en el rendimiento físico del servidor. Cuenta con una arquitectura basada en **Instrumentos** (código fuente monitorizado) y **Consumidores** (tablas del motor de almacenamiento en memoria que recopilan las métricas).

- **events_statements_summary_by_digest:** Agrupa estadísticas sobre patrones de consultas (digests), facilitando identificar cuáles consumen más CPU o tiempo de ejecución.
- **threads:** Muestra información técnica detallada sobre los hilos de ejecución activos del servidor.
- **events_errors_summary_by_user_by_error:** Registra resúmenes históricos de los errores de sintaxis o conexión provocados por cada usuario del sistema.

Caso Práctico 4: Listar los errores más comunes provocados por los usuarios

```
SELECT
user AS 'Usuario',
error_number AS 'Código de Error',
error_name AS 'Nombre de Error',
sum_error_raised AS 'Veces provocado',
first_seen AS 'Primera detección',
last_seen AS 'Último registro'
FROM performance_schema.events_errors_summary_by_user_by_error
WHERE sum_error_raised > 0
ORDER BY sum_error_raised DESC;
```

3.4. Base de Datos sys (La Abstracción Simplificada)

La complejidad de performance_schema dificulta su uso rutinario en el día a día del SysAdmin. Para solucionar esto, MySQL incluye la base de datos sys, que agrupa vistas estructuradas de forma intuitiva, funciones y procedimientos predefinidos.

Caso Práctico 5: Identificar las consultas SQL que provocan mayor latencia

```
SELECT
query AS 'Sentencia SQL',
db AS 'Base de Datos',
exec_count AS 'Total Ejecuciones',
total_latency AS 'Tiempo Total acumulado',
max_latency AS 'Peor latencia registrada'
FROM sys.statement_analysis
```

```
ORDER BY total_latency DESC
LIMIT 5;
```

Caso Práctico 6: Encontrar tablas con lecturas completas

La monitorización sistemática de consultas que no aprovechan los índices evita problemas de degradación del disco físico y ahorra energía en el centro de datos.

```
SELECT
schemaname AS 'Base de Datos',
tablename AS 'Tabla',
scans AS 'Lecturas Completas',
rows_sel AS 'Filas devueltas'
FROM sys.schema_tables_with_full_table_scans
ORDER BY scans DESC;
```

4. Automatización Mediante Scripts Externos (Bash)

La verdadera automatización de la administración ocurre cuando conectamos el SGBD con el Sistema Operativo utilizando scripts de consola (por ejemplo, Bash en sistemas Linux).

Para que un script de Bash interactúe con MySQL de forma desasistida, solemos utilizar el cliente de línea de comandos de MySQL mediante la estructura:

mysql -u "usuario" -p"contraseña" -e "CONSULTA_SQL"

Buenas Prácticas de Seguridad: Exponer la contraseña directamente en el script utilizando -p"mi_password" es una vulnerabilidad grave (cualquier usuario del sistema podría verla ejecutando ps aux).

Recomendación: Guardar las credenciales en un archivo `.env` con permisos 600 y cargarlo desde el propio script con `source .env`. De este modo se evita dejar la contraseña en el historial del sistema o en el código fuente.

```
# .env (guardar en la carpeta del proyecto y protegerlo: chmod 600 .env)
MYSQL_USER=root
```

```
MYSQL_PASSWORD='PasswordMuySegura123!'
MYSQL_DATABASE=tienda_informatica
BACKUP_DIR="/var/backups/mysql"
```

5. Ejemplos Prácticos de Scripts

Para asentar estos conceptos, a continuación se presentan tres scripts sencillos para administración de sistemas Linux/MySQL. Están diseñados bajo criterios didácticos, con un nivel de complejidad progresivo y comentarios exhaustivos de cada instrucción.

Script 1: Comprobación de Salud del Servidor (Health Check)

Este script verifica si el servicio MySQL está activo en el sistema operativo y si responde a consultas básicas. Si falla, genera una alerta visual y un registro en el log del sistema.

```
#!/bin/bash
set -euo pipefail

# Cargar variables seguras desde .env
if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] No existe el archivo .env. Crea uno con MYSQL_USER y MYSQL_PASSWORD." >&2
  exit 1
fi

# Seguridad: evitar que el archivo .env sea visible por otros usuarios
umask 077

FECHA_LOG=$(date "+%Y-%m-%d %H:%M:%S")

echo "=== INICIANDO CONTROL DE SALUD DEL SGBD ($FECHA_LOG) ==="

if ! systemctl is-active --quiet mysql; then
  echo "[ALERTA] MySQL está caído. Intentando arrancarlo..."
  sudo systemctl start mysql
  sleep 3
fi

if MYSQL_PWD="$MYSQL_PASSWORD" mysqladmin --user="$MYSQL_USER" ping &>/dev/null; then
```

```
  echo "[OK] MySQL está levantado y responde correctamente."
else
  echo "[CRÍTICO] MySQL está activo pero no acepta conexiones."
  exit 1
fi

echo "=== CONTROL DE SALUD FINALIZADO ==="
exit 0
```

Script 2: Copia de Seguridad Sencilla (Backup Automatizado)

Genera un respaldo (.sql) de una base de datos específica, le asigna una marca de tiempo para no sobreescribir copias anteriores y comprime el archivo para ahorrar espacio en disco.

```
#!/bin/bash
set -euo pipefail

if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] Falta el archivo .env" >&2
  exit 1
fi

mkdir -p "$BACKUP_DIR"
chmod 700 "$BACKUP_DIR"

MARCA_TIEMPO=$(date "+%Y%m%d_%H%M%S")
FICHERO_BACKUP="$BACKUP_DIR/backup_${MYSQL_DATABASE}_${MARCA_TIEMPO}.sql"

echo "Iniciando copia de seguridad de $MYSQL_DATABASE..."

MYSQL_PWD="$MYSQL_PASSWORD" mysqldump \
  --user="$MYSQL_USER" \
  --databases "$MYSQL_DATABASE" \
  --single-transaction \
  --routines \
  --events \
  > "$FICHERO_BACKUP"

gzip -f "$FICHERO_BACKUP"
```

```
echo "Copia creada correctamente en ${FICHERO_BACKUP}.gz"
exit 0
```

Script 3: Comprobación de Consumo y Limpieza de Logs

Un administrador debe controlar el tamaño de la base de datos. Este script consulta el catálogo information_schema para verificar si el tamaño supera un límite seguro y limpia una tabla histórica si es necesario para liberar espacio.

```
#!/bin/bash
set -euo pipefail

if [ -f ".env" ]; then
  set -a
  source .env
  set +a
else
  echo "[ERROR] Falta el archivo .env" >&2
  exit 1
fi

LIMITE_MB=500
QUERY_TAMAÑO="SELECT ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) FROM information
_schema.tables;"

echo "Consultando el tamaño total ocupado en disco por el SGBD..."
TAMANO_ACTUAL=$(MYSQL_PWD="$MYSQL_PASSWORD" mysql --user="$MYSQL_USER" -AN -e "$QUERY_TAMAÑ
O")
TAMANO_ENTERO=${TAMANO_ACTUAL%.*}

echo "Tamaño de datos actual en el servidor: $TAMANO_ACTUAL MB"

if [ "$TAMANO_ENTERO" -gt "$LIMITE_MB" ]; then
  echo "[AVISO] Se ha superado el umbral seguro de $LIMITE_MB MB."
  echo "[MANTENIMIENTO] Procediendo a purgar datos antiguos..."

  QUERY_PURGA="DELETE FROM logs_sistema.registro_eventos WHERE fecha_evento < NOW() - INTERVA
L 30 DAY;"
  MYSQL_PWD="$MYSQL_PASSWORD" mysql --user="$MYSQL_USER" -e "$QUERY_PURGA"

  echo "Purga de registros completada."
else
  echo "[INFORMACIÓN] El espacio ocupado está dentro de los rangos óptimos."
fi
```

```
exit 0
```